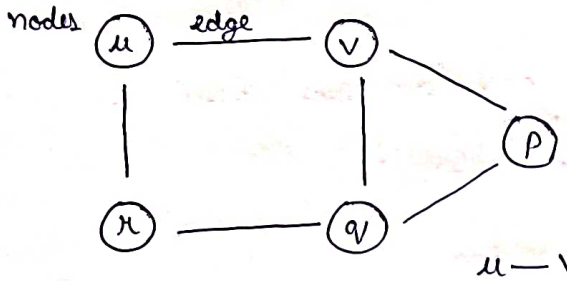
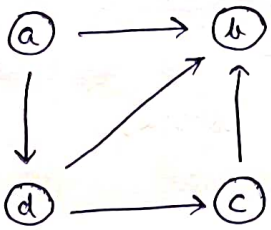
Graphs → is a data structure which is combination of nodes & edges



u — v ✓
v — u ✓

Undirected



a — b ✓
b — a ✗

Directed

Node → entity to store data

edge → for connecting nodes

Degree (v) = 3

Degree (u) = 2

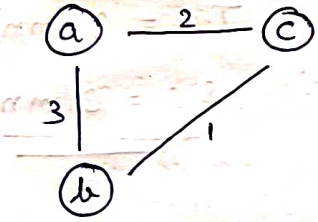
Indegree (b) = 3

Outdegree (b) = 0

Indegree (a) = 0

Outdegree (a) = 2

Weighted graph



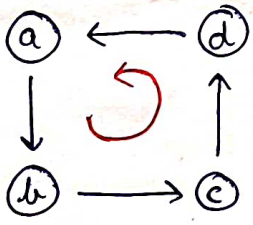
a — c ⇒ 2
a — b ⇒ 3
b — c ⇒ 1
} Weights

Agar weight nhi hai to 1 assume kar lo

Path : u — v — p ✓
p — q — v ✓
p — q — v — p ✗

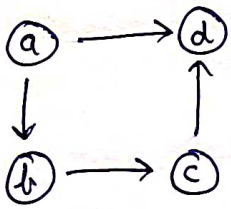
a — d — c — b ✓
a — d — b ✓

Cyclic graph



a → b → c → d

Acyclic graph



a — b — c — d

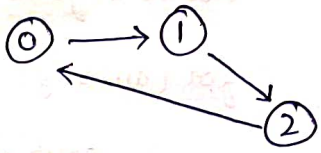
Representation:-

① Adjacency Matrix

i/p ⇒ no of nodes
no of edges
list of edges

Ex:- i/p ⇒ n = 3, m = 3

0 — 1
1 — 2
2 — 0



Representation:-

2D matrix

	0	1	2
0	0	1	0
1	0	0	1
2	1	0	0

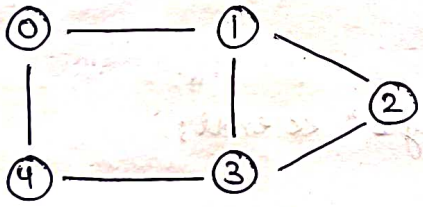
index denotes the nodes

S.C. = $O(n^2)$

② Adjacency list

~~Representation for same example:-~~

i/p ⇒ n = 5, m = 6



Representation:-

0 → 1, 4
1 → 0, 2, 3
2 → 1, 3
3 → 1, 2, 4
4 → 0, 3

} Adjacency list

Implementation:-

map<int, list<int>>

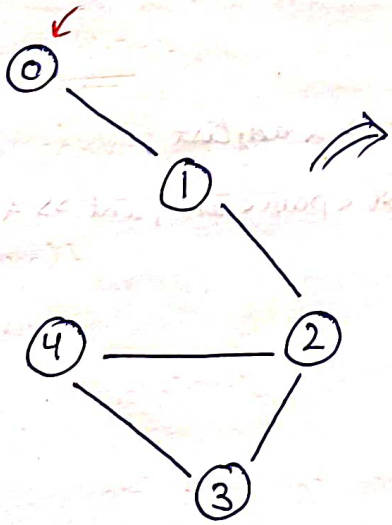
```
class graph {
    public:
        unordered_map<int, list<int>> adj;

        void addEdge(int u, int v, bool direction) {
            adj[u].push_back(v);
            if (direction == 0) {
```

```
            adj[v].push_back(u);
        }
    }

    void printAdjList() {
        for(auto i: adj) {
            cout << i.first << " ->";
            for (auto j: i.second) {
                cout << j << ", ";
            }
            cout << endl;
        }
    }
};

int main() {
    int n;
    cout << "Enter the number of nodes" << endl;
    cin >> n;

    int m;
    cout << "Enter the number of edges" << endl;
    cin >> m;

    graph g;
    for (int i=0; i<m; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v, 0);    // Creating undirected graph
    }

    g.printAdjList();
}
```

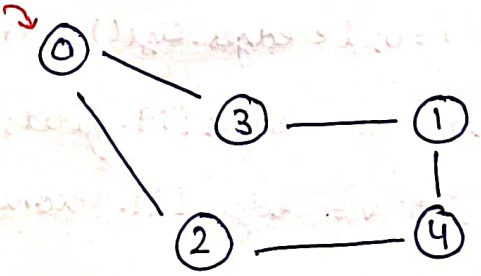
Adjacency list using vector<vector<int>>

```
vector<vector<int>> printAdjacency(int n, int m, vector<vector<int>>& edges) {
    vector<int> ans[n];
    for (int i=0; i<m; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        ans[u].push_back(v);
        ans[v].push_back(u);
    }
    vector<vector<int>> adj(n);
    for (int i=0; i<n; i++) {
        adj[i].push_back(i);
        for (int j=0; j<ans[i].size(); j++) {
            adj[j].push_back(ans[i][j]);
        }
    }
    return adj;
}
```

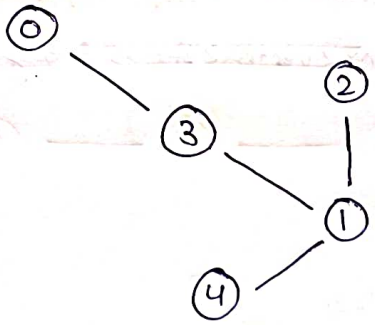
Breadth First Search



Traversal ⇒ 0, 1, 2, 3, 4



Traversal ⇒ 0, 2, 3, 4, 1



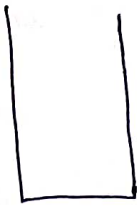
visited Data Structure

unordered_map < node, bool >;
all nodes false initially

adj. list

0 ⇒ 3
1 ⇒ 2, 3, 4
2 ⇒ 1
3 ⇒ 0, 1
4 ⇒ 1

```
if ( !visited[node] ) {
    bfs();
}
```



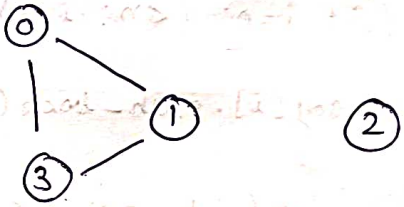
queue

Algo:-

① q.push( );
② frontNode = q.front();
③ q.pop();
④ vis[node] = true; & print node
⑤ push neighbours of that node in q

T.C = O(n+E)
S.C = O(n+E)

If you want ans in sorted order then use set instead of list



component component

Disconnected graph

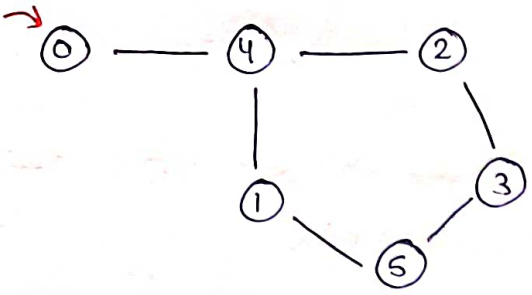
For this case:
if ke upar loop laga de

```
for (int i=0; i<n; i++) {
    if (!visited[node]) {
        bfs();
    }
}
```

```
void prepareAdjList(unordered_map<int, list<int>> & adjList,
                    vector<pair<int, int>> & edges) {
    for (int i=0; i<edges.size(); i++) {
        int u = edges[i].first;
        int v = edges[i].second;
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }
}
```

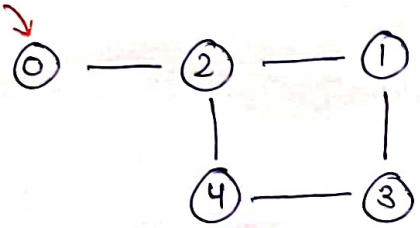
```
void bfs (unordered_map<int, list<int>> & adjList, unordered_map<int, bool>
                & visited, vector<int> & ans, int node) {

    queue<int> q;
    q.push(node);
    visited[node] = 1;

    while (! q.empty()) {

        int frontNode = q.front();

        q.pop();

        ans.push_back(frontNode);

        for (auto i : adjList[frontNode]) {

            if (! visited[i]) {

                q.push(i);

                visited[i] = 1;

            }

        }

    }

}

vector<int> BFS(int vertex, vector<pair<int, int>> edges) {

    unordered_map<int, list<int>> adjList;

    vector<int> ans;

    unordered_map<int, bool> visited;

    prepareAdjList (adjList, edges);

    for (int i=0; i<vertex; i++) {

        if (! visited[i]) {

            bfs (adjList, visited, ans, i);

        }

    }

    return ans;

}
```

Depth First Search



BFS :- 0, 4, 2, 1, 3, 5

DFS :- 0, 4, 2, 3, 5, 1



BFS :- 0, 2, 1, 4, 3

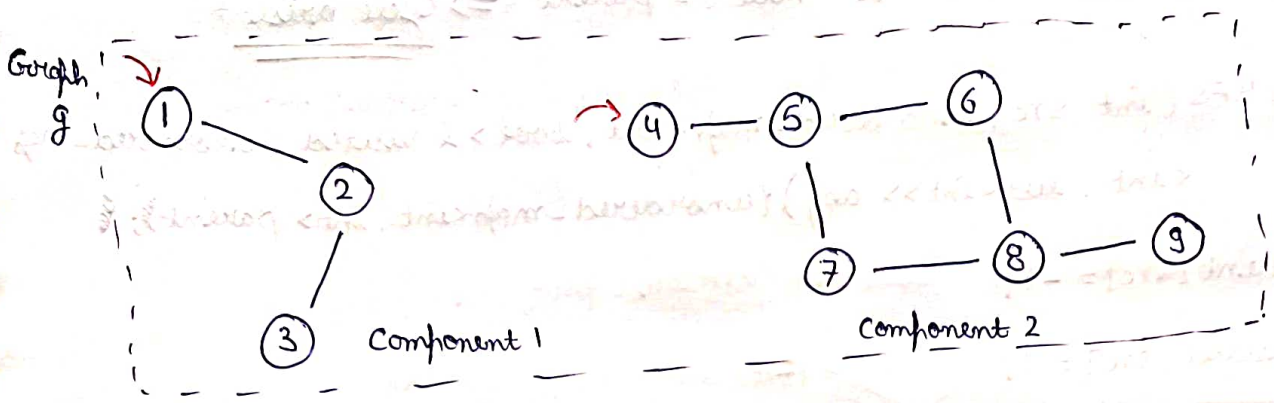
DFS :- 0, 2, 1, 3, 4

```
void dfs(int node, unordered_map<int, bool> & visited, unordered_map<int,
                list<int>> & adj, vector<int> & component) {
    component.push_back(node);
    visited[node] = true;
    for(auto i: adj[node]) {
        if(!visited[i]) {
            dfs(i, visited, adj, component);
        }
    }
}

vector<vector<int>> depthFirstSearch(int V, vector<vector<int>> & edges) {
    unordered_map<int, list<int>> adj;
    vector<vector<int>> ans;
    prepareAdjList(adj, edges);
    unordered_map<int, bool> visited;
    for(int i=0; i<V; i++) {
        if(!visited[i]) {
            vector<int> component;
            dfs(i, visited, adj, component);
        }
        ans.push_back(component);
```

return ans;

}

Qu- Detect cycle in an undirected graph



From BFS

adjList

1 → 2

2 → 3, 1

3 → 2

4 → 5

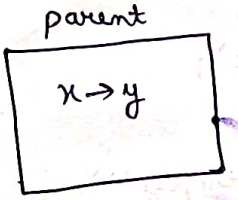
5 → 4, 6, 7

6 → 5, 8

7 → 5, 8

8 → 6, 7, 9

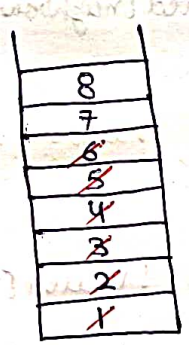
9 → 8



neglect

↳ visited & parent

queue



Dry Run:-

vis[1] = 1

1 → -1 (Parent)

q.push(1)

push neighbours of 1 & mark vis

vis[2] = 1

2 → 1 (Parent)

push neighbours of 2 & mark vis

vis[3] = 1

3 → 2 (Parent)

src=4

4 → -1 (Parent)

vis[4] = 1

push neighbours of 4 & mark vis

vis[5] = 1

5 → 4 (Parent)

push neighbours of 5 & mark vis

vis[6] = 1

6 → 5 (Parent)

vis[7] = 1

7 → 5 (Parent)

push neighbours of 6 & mark vis

vis[8] = 1

8 → 6 (Parent)

Now, push neighbours of 7 and mark vis

8 is already vis but 8 is not a parent of 7 i.e. Cycle present

So, if vis[node] = True & node != parent ⇒ Cycle present

```
bool isCyclicBFS(int src, unordered_map<int, bool> & visited, unordered_map
        <int, list<int>> adj){unordered_map<int, int> parent};{
    parent[src] = -1;
    visited[src] = 1;
    queue<int> q;
    q.push(src);

    while(!q.empty()) {
        int front = q.front();
        q.pop();

        for(auto neighbour: adj[front]) {

            if(visited[neighbour] == true && neighbour !=
                                                parent[front]){
                return true;
            }

            else if(!visited[neighbour]) {
                q.push(neighbour);
                visited[neighbour] = 1;
                parent[neighbour] = front;
            }
        }
    }
    return false;
}
```

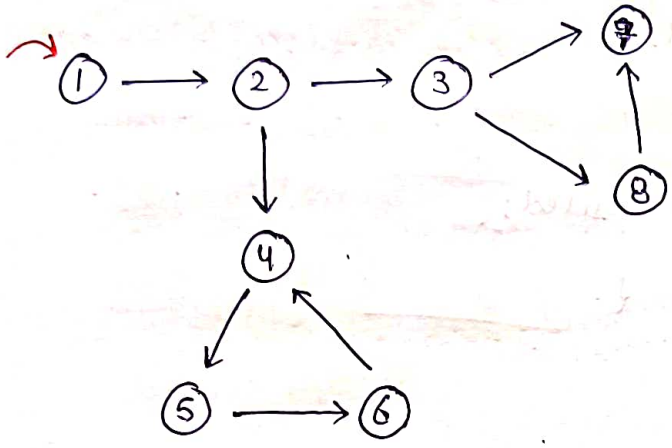
```
String cycleDetection (vector<vector<int>> & edges, int n, int m) {
    unordered_map<int, list<int>> adj;
    for (int i=0; i<m; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    unordered_map<int, bool> visited;
    for (int i=0; i<n; i++) {
        if (!visited[i]) {
            bool ans = isCyclicBFS(i, visited, adj);
            if (ans == 1) {
                return "Yes";
            }
        }
    }
    return "NO";
}
```

From DFS (same logic of parent)

```
bool isCyclicDFS (int node, int parent, unordered_map<int, bool> & visited,
                  unordered_map<int, list<int>> &adj) {
    visited[node] = true;
    for (auto it: adj[node]) {
        if (!visited[it]) {
            bool cycleDetected = isCyclicDFS(it, node, visited, adj);
            if (cycleDetected)
                return true;
        }
```

```
        else if( it != parent) {
            return true;
        }
    }
    return false;
}
```

Qu- Cycle Detection in Directed Graphs



Using DFS

adj list

1 → 2

2 → 3, 4

3 → 7, 8

4 → 5

5 → 6

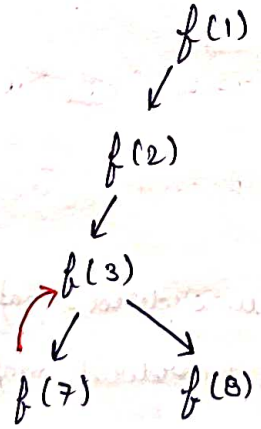
6 → 4

7 →

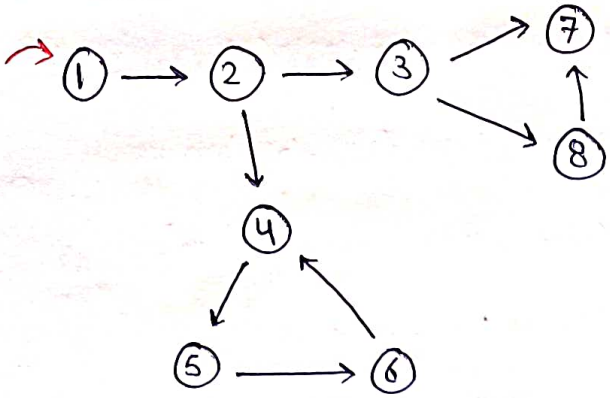
8 → 7

visited

1	2	3	4	5	6	7	8
~~0~~ 1	~~0~~ 1	~~0~~ 1	0	0	0	~~0~~ 1	~~0~~ 1



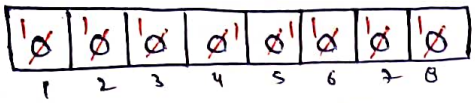
neighbour of 8 is 7 but 7 is already vis & 8 is not a parent of 7 as well but it is not a cycle so this condn which we have applied in case of undirected graph is not valid here

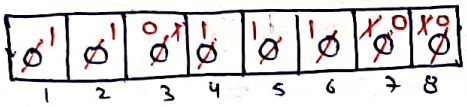


adj List

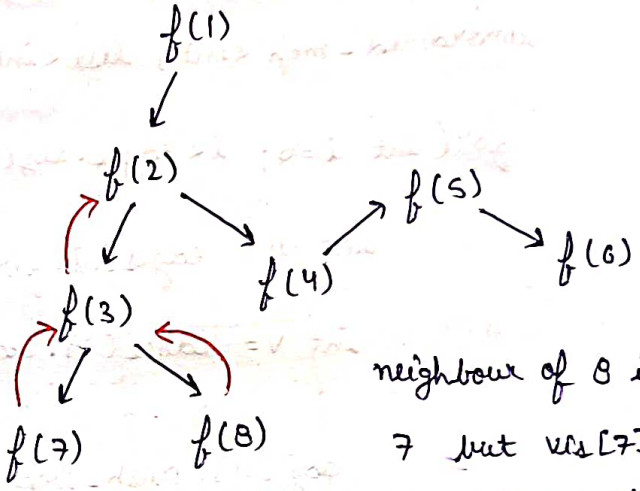
1 → 2
2 → 3, 4
3 → 7, 8
4 → 5
5 → 6
6 → 4
7 →
8 → 7

visited





dfs visited



neighbour of 8 is 7 but vis[7] = 1 so, 7 ki call nhi jaegi

neighbour of 6 is 4 & vis[4] = 1 && dfs vis[4] = 1 also so cycle is present

So, cond^n for cyclic:-

vis[node] == 1 && dfs[vis[node] == 1

T.C = O(N+E)

S.C. = Linear

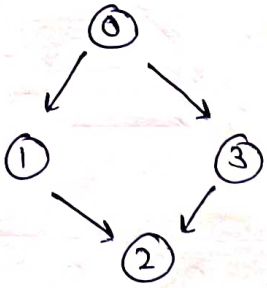
```
bool checkCycleDFS(int node, unordered_map<int, bool>& vis,
        unordered_map<int, bool>& dfsvis, unordered_map<int, list<int>>
        &adj) {

    vis[node] = true;
    dfsvis[node] = true;

    for(auto i : adj[node]) {
        if(!vis[i]) {
            bool cycleDetected = checkCycleDFS(i, vis, dfsvis, adj);
            if(cycleDetected) {
                return true;
            }
        }
    }
```

```
        else if (dfsVis[i]) {
            return true;
        }
    }
    dfsVis[node] = false;
    return false;
}

int detectCycleInDirectedGraph(int n, vector<pair<int, int>>& edges) {
    unordered_map<int, list<int>> adj;
    for(int i=0; i<edges.size(); i++) {
        int u = edges[i].first;
        int v = edges[i].second;
        adj[u].push_back(v);
    }
    unordered_map<int, bool> vis;
    unordered_map<int, bool> dfsVis;
    for(int i=1; i<=n; i++) {
        if (!visited[i]) {
            bool cycleFound = checkCycleDFS(i, vis, dfsVis, adj);
            if (cycleFound) {
                return true;
            }
        }
    }
    return false;
}
```

Qu- Topological sort (Linear ordering of vertices such that for every edge $u \to v$, u always appears before v in that ordering.

Only valid in case of DAG (Directed Acyclic Graph)

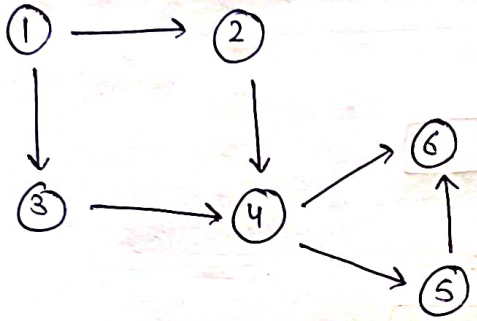
Ex:-



o/p ⇒ 0 1 3 2

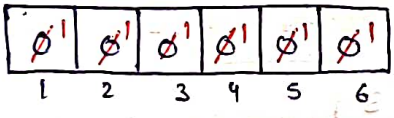
0 → 1, 3
1 → 2
2 →
3 → 2

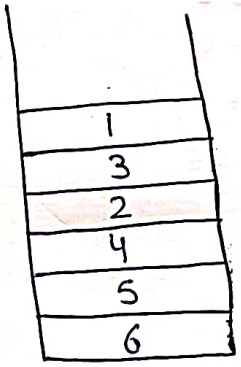
Using DFS



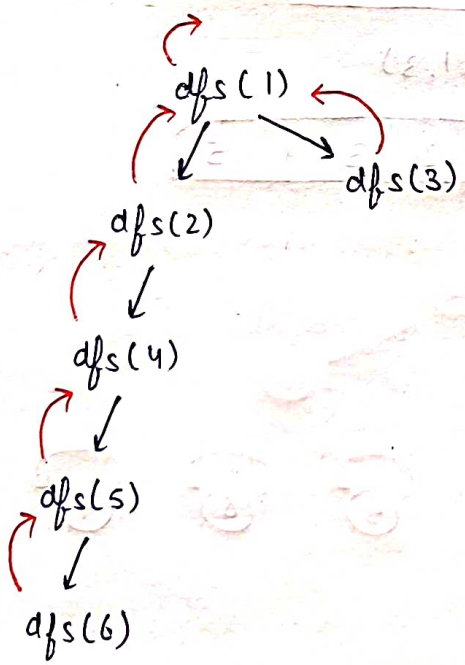
adj List

1 → 2, 3
2 → 4
3 → 4
4 → 5, 6
5 → 6
6 →





Stack (LIFO)



Now stack se nikaal kar jo order bana :-

1, 3, 2, 4, 5, 6

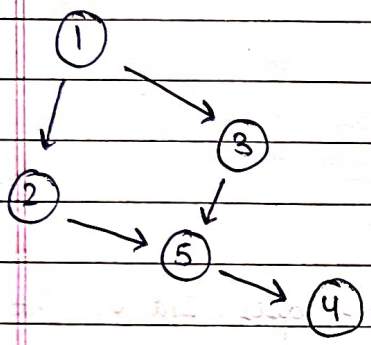
Valid Topological sort

1

```
void topoSort(int node, vector<bool> & vis, stack<int> & s,
              unordered_map<int, list<int>> & adj) {
    vis[node] = 1;
    for (auto i: adj[node]) {
        if (!vis[i]) {
            topoSort(i, vis, s, adj);
        }
    }
    s.push(node);
}
vector<int> topologicalSort(vector<vector<int>> & edges, int v, int e) {
    unordered_map<int, list<int>> adj;
    for (int i=0; i<e; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        adj[u].push_back(v);
    }
    vector<bool> vis(v);
    stack<int> s;
    for (int i=0; i<v; i++) {
        if (!vis[i]) {
            topoSort(i, vis, s, adj);
        }
    }
```

```
vector<int> ans;
while (!s.empty()) {
    ans.push_back(s.top());
    s.pop();
}
return ans;
}
```

Using Kahn's algorithm



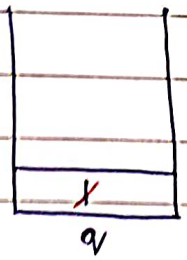
① Find indegree of all nodes

	1	2	3	4	5
Indegree	0	1	1	1	2

② Make queue & insert all nodes having Indegree 0

adjlist:
1 → 2, 3
2 → 5
3 → 5
4 →
5 → 4

③ Do BFS

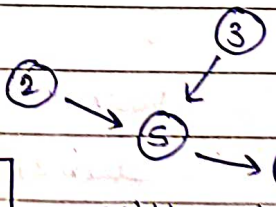


front = 1

ans | 1 |

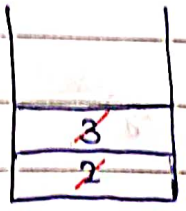
	1	2	3	4	5
Indegree	0	0	0	1	2

Now graph will look like



neighbour of 1 → 2, 3

Now insert all nodes having indegree 0



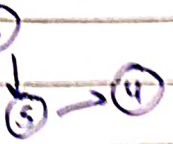
front = 2

ans = | 1 | 2 |

	1	2	3	4	5
Indegree	0	0	0	1	1

neighbours of 2 → 5

so, indegree of 5 = 1



front = 3

neighbour of 3 → 5

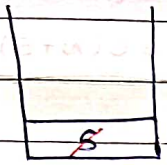
q.pop()

ans ⇒ | 1 | 2 | 3 |

(5) → (4)

indegree ⇒ | 0 | 0 | 0 | 1 | 0 |
1 2 3 4 5

↳ q.push(5)



front = 5

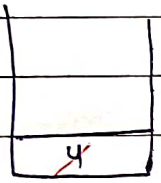
neighbour of 5 → 4

q.pop()

ans ⇒ | 1 | 2 | 3 | 5 |

indegree ⇒ | 0 | 0 | 0 | 0 | 0 |
1 2 3 4 5

↳ q.push(4)



front = 4

4 has no neighbours

q.pop()

ans ⇒ | 1 | 2 | 3 | 5 | 4 |

```
vector<int> topological sort(vector<vector<int>> & edges, int v, int e) {

    unordered_map<int, list<int>> adj;

    for(int i=0; i<e; i++) {

        int u = edges[i][0];
        int v = edges[i][1];
        adj[u].push_back(v);

    }

    vector<int> indegree(v);
    for(auto i:adj) {
        for(auto j: i.second) {

            indegree[j]++;

        }

    }

    queue<int> q;
```

```
for ( int i=0; i<v; i++) {
    if (indegree[i]==0) {
        q.push(i);
    }
}

vector<int> ans;
while (!q.empty()) {
    int front = q.front();
    q.pop();
    ans.push_back(front);
    for(auto i : adj[front]) {
        indegree[i]--;
        if (indegree[i]==0) {
            q.push(i);
        }
    }
}
return ans;
}
```

T.C. = O(N+E)

S.C. = O(N+E)

Qu- Cycle Detection in directed graph (Using BFS) (2)

Topological sort can only be true in case of Directed Acyclic Graph

So, we will find out the topological sort of the graph and if it is incorrect that means cycle is present.

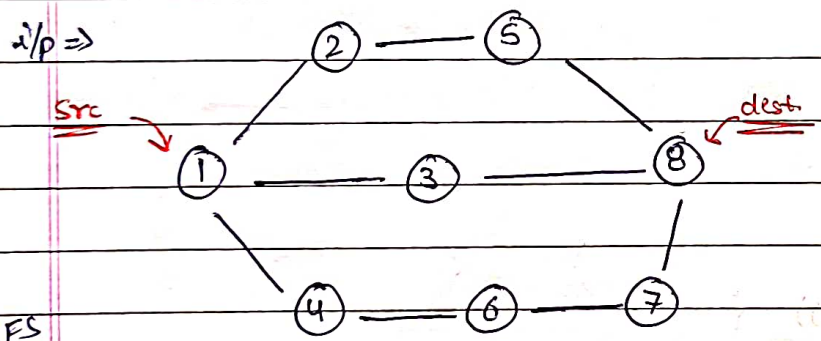
Kahn's algorithm code

⇒ To iske code me bas ham while loop me count ko value nikaal lenge aur if count != no. of nodes then return true

T.C. = O(N+E)

S.C. = O(N+E)

Qu- Shortest path in undirected graph



o/p ⇒ | 1 | 3 | 8 |

BFS

adj list

1 → 2, 3, 4

2 → 1, 5

3 → 1, 8

4 → 1, 6

5 → 2, 8

6 → 4, 7

7 → 6, 8

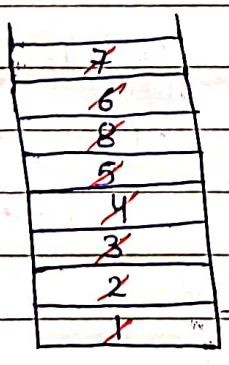
8 → 3, 5, 7

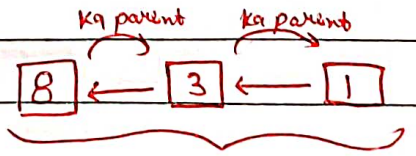
vis

1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0

parent

1	2	3	4	5	6	7	8
-1	1	1	1	2	4	8	3





Reverse & return

(Backtrack to parent)

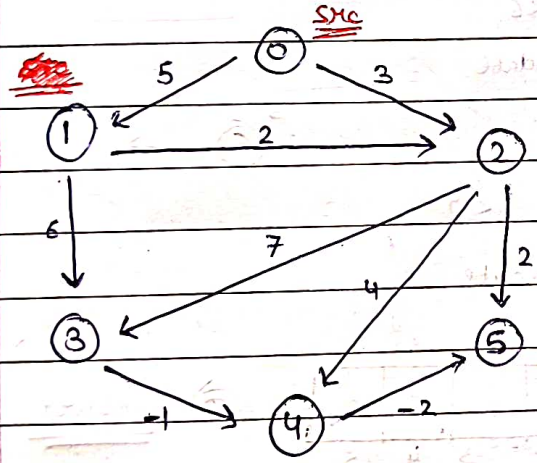
```
vector<int> shortestPath (vector<pair<int, int>> edges, int n, int m, int s, int t) {
    unordered_map<int, list<int>> adj;
    for (int i=0; i< edges.size(); i++) {
        int u = edges[i].first;
        int v = edges[i].second;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    unordered_map<int, bool> vis;
    unordered_map<int, int> parent;
    queue<int> q;
    q.push(s);
    parent[s] = -1;
    vis[s] = true;
    while (!q.empty()) {
        int front = q.front();
        q.pop();
        for (auto i: adj[front]) {
            if (!vis[i]) {
                vis[i] = true;
                parent[i] = front;
                q.push(i);
            }
        }
    }
```

```
vector<int> ans;
int currentNode = t;
ans.push_back(t);
while (currentNode != s) {
    currentNode = parent[currentNode];
    ans.push_back(currentNode);
}
reverse(ans.begin(), ans.end());
return ans;
}
```

T.C. = O(N+E)

S.C. = O(N+E)

Qu- Shortest path in Directed Acyclic Graph



Shortest distance

1 → 0 ⇒ Infinite

1 → 1 ⇒ 0

1 → 2 ⇒ 2

1 → 3 ⇒ 6

1 → 4 ⇒ 5

1 → 5 ⇒ 3

Shortest path ⇒ { INF, 0, 2, 6, 5, 3 } (for 1)

Algo:-

① Topological sort find

② Linear ordering → distance array update

adj list unordered_map<int, list<pair<int, int>>> adj

0 → [1, 5], [2, 3]

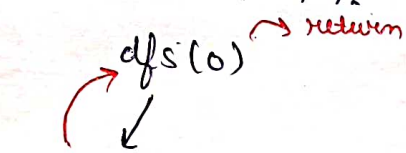
1 → [2, 2], [3, 6]

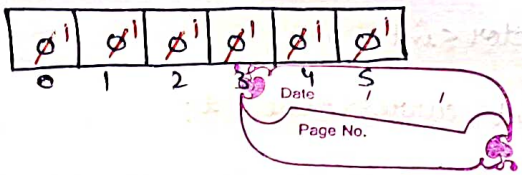
2 → [3, 7], [4, 4], [5, 2]

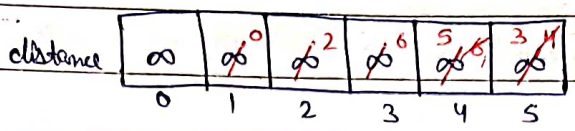
3 → [4, -1]

4 → [5, -2]

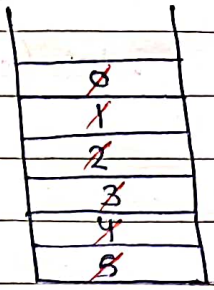
5 →



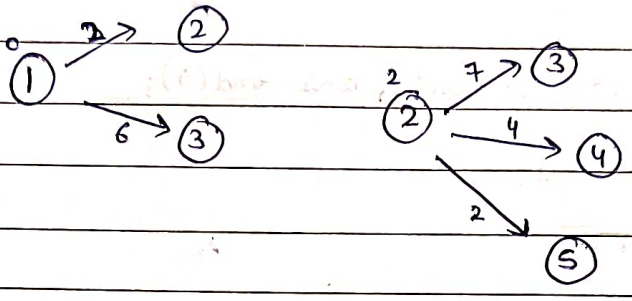




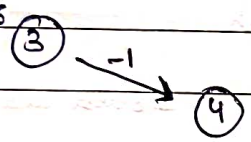
Taking 1 as src node



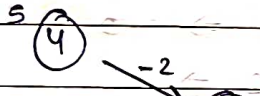
Stack<int> s



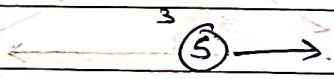
2→3 : 9 but 6 < 9
So we don't update



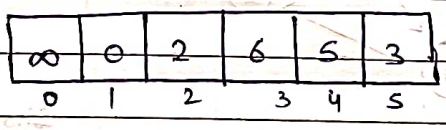
3→4 : 5 & 5 < 6
so, update



4→5 : 3 & 3 < 4
so, update



Now dist. array ⇒



answer

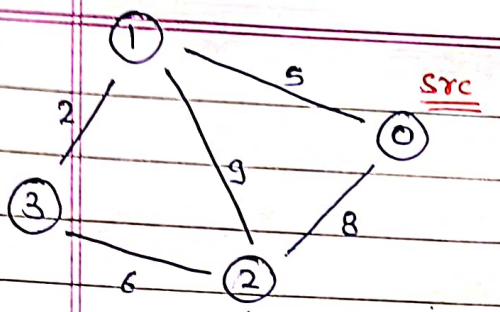
```
class Graph {
public:
    unordered_map<int, list<pair<int, int>>> adj;

    void addEdge(int u, int v, int weight) {
        pair<int, int> p = make_pair(v, weight);
        adj[u].push_back(p);
    }
};
```

```
void dfs (int node, unordered_map<int, bool> & vis, stack<int> & topo) {
    vis[node] = true;
    for (auto i : adj[node]) {
        if (!vis[i.first]) {
            dfs (i.first, vis, topo);
        }
    }
    topo.push (node);
}

main() {
    int src = 1;
    vector<int> dist(n);
    for (int i=0; i<n; i++) {
        dist[i] = INT_MAX;
    }
    dist[src] = 0;
    while (!s.empty()) {
        int top = s.top();
        s.pop();
        for (auto x:
        if (dist[top] != INT_MAX) {
            for (auto i : adj[top]) {
                if (dist[top] + i.second < dist[i.first]) {
                    dist[i.first] = dist[top] + i.second;
                }
            }
        }
    }
}
```

T.C. = O(N+E)

S.C. = O(N+E)

Dijkstra's Shortest Path



src

adj list

0 → [1, 5], [2, 8]
1 → [0, 5], [2, 9], [3, 2]
2 → [0, 8], [1, 9], [3, 6]
3 → [1, 2], [2, 6]

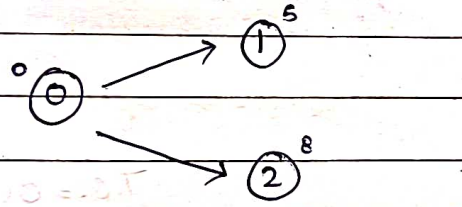
Shortest dist.

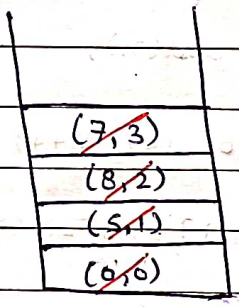
0 → 1	5
0 → 2	8
0 → 3	7
0 → 0	0

⇒ 0, 5, 8, 7 answer

dist[src] = 0

dist	∞ 0	∞ 5	∞ 8	∞ 7
	0	1	2	3

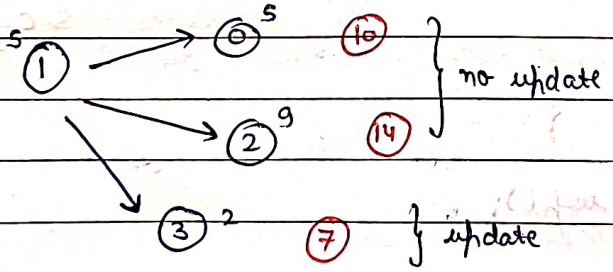




Set

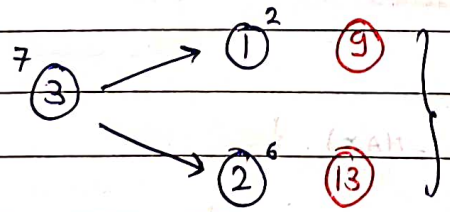
In set top node is that which is having minimum distance

pair<int, int>
dist. node

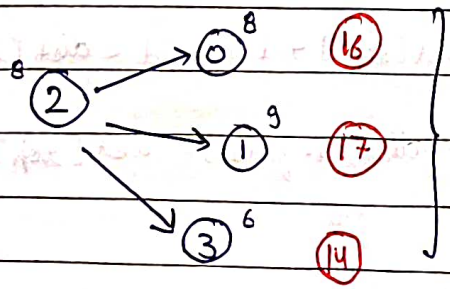


no update

update

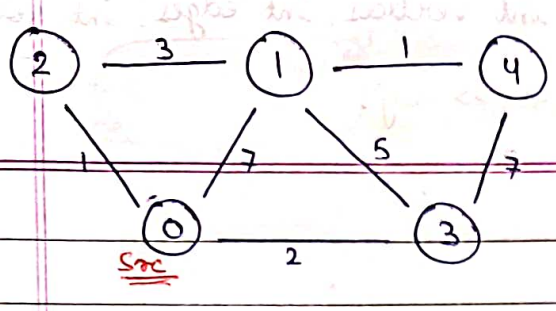


no update



no update

So, dist. final array is the answer



adj list

0 → [1,7], [2,1], [3,2]

1 → [0,7], [2,3], [3,5], [4,1]

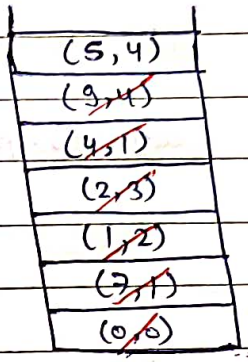
2 → [0,1], [1,3]

3 → [0,2], [1,5], [4,7]

4 → [1,1], [3,7]

dist[src] = 0

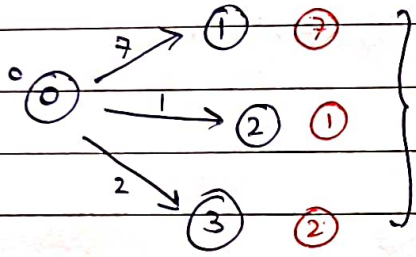
0	1	2	3	4
~~∞~~ 0	~~∞~~ 4	~~∞~~ i	~~∞~~ 2	~~∞~~ 5

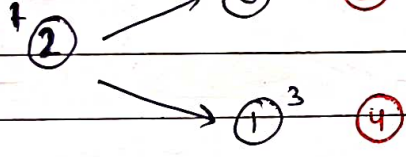


Set

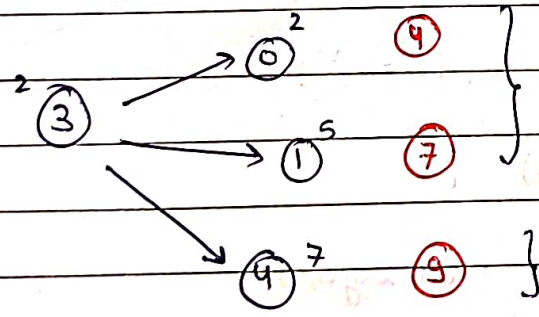
pair<int, int>

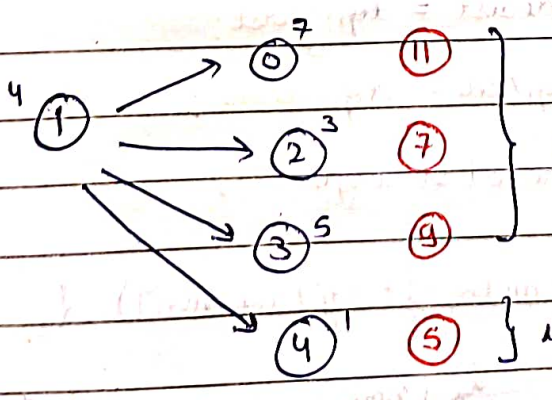
dist. node



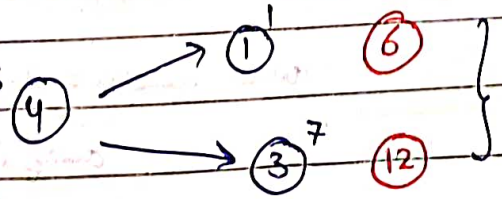


& update entry in set (7,1) to (4,1)





& entry in set (9,4) to (5,4)



dist. ⇒ 0, 4, 1, 2, 5

answer

```
vector<int> dijkstra(vector<vector<int>> &vec, int vertices, int edges, int source) {
unordered_map<int, list<pair<int, int>>> adj;

for (int i=0; i<edges; i++) {
    int u = vec[i][0];
    int v = vec[i][1];
    int w = vec[i][2];
    adj[u].push_back(make_pair(v, w));
    adj[v].push_back(make_pair(u, w));
}

vector<int> dist(vertices);
for(int i=0; i<vertices; i++)
    dist[i] = INT_MAX;

set<pair<int, int>> st;
dist[source] = 0;
st.insert(make_pair(0, source));

while (!st.empty()) {
    auto top = *(st.begin());
    int nodeDist = top.first;
    int topNode = top.second;
    st.erase(st.begin());

    for (auto i: adj[topNode]) {
        if (nodeDist + i.second < dist[i.first]) {
            auto record = st.find(make_pair(
                dist[i.first], i.first));
```

T.C. = O(E log V)

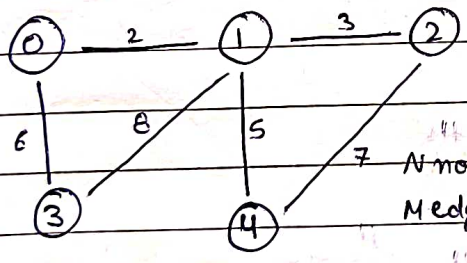
S.C. = O(N+E)

(4)

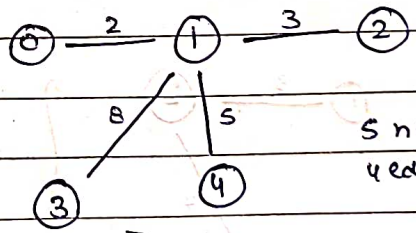
```
if (record != st.end()) {
    st.erase(record);
}
}
dist[i.first] = nodeDist + i.second;
st.insert(make_pair(dist[i.first], i.first));
}
}
}
return dist;
}
```

Prim's Algorithm

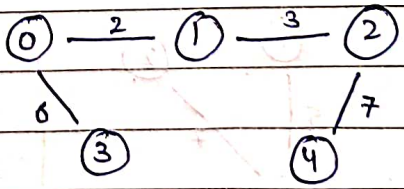
If you can convert a graph into a tree having n nodes and n-1 edges then it is called spanning Tree. and also every node is reachable by any other node.



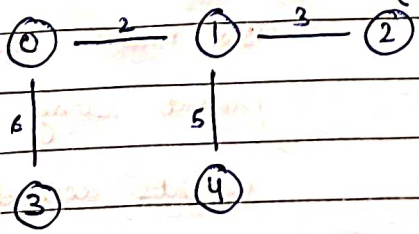
Graph



Spanning Tree

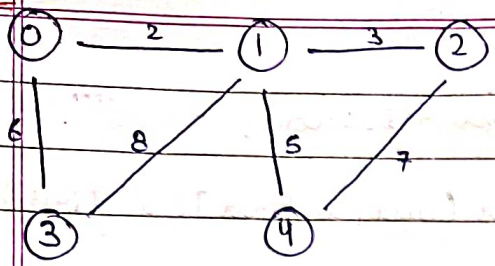


Minimum Spanning Tree
i.e. (minimum cost of weights)



Prim's Algo

Src



key

0	∞ 2	∞ 3	∞ 6	∞ 5
0	1	2	3	4

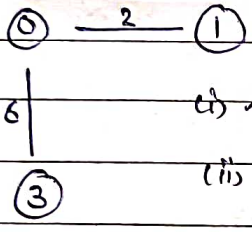
mst

F T	F T	F T	F T	F T
0	1	2	3	4

① Find min. in key having F in mst parent

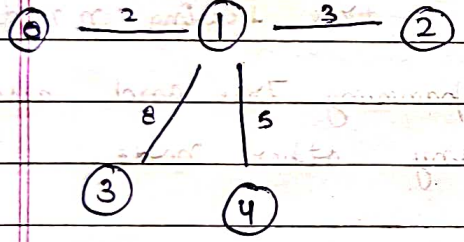
② Mark True in mst for min.

-1	-1 0	-1 1	-1 0	-1 1
0	1	2	3	4

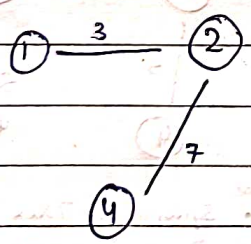


(i) update distance in key

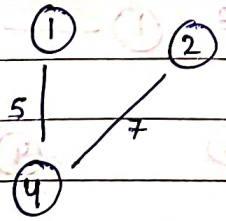
(ii) mark parent of neighbours jisko update kiya tha



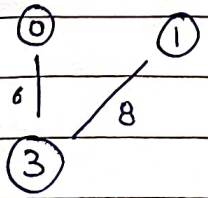
update in key for 2 & 4
& mark parents of 2 & 4



no updation



no updation



no updation

Now make tree from parent array and set weights according to key array

```
vector<pair<pair<int,int>,int>> calculatePrimsMST(int n, int m, vector<pair<
                                    pair<int,int>,int>> g) {
unordered_map<int, list<pair<int,int>>> adj;

for(int i=0; i<g.size(); i++) {
    int u = g[i].first.first;
    int v = g[i].first.second;
    int w = g[i].second;
    adj[u].push_back(make_pair(v,w));
    adj[v].push_back(make_pair(u,w));
}

vector<int> key(n+1);
vector<bool> mst(n+1);
vector<int> parent(n+1);
for(int i=0; i<=n; i++) {
    key[i] = INT_MAX;
    parent[i] = -1;
    mst[i] = false;
}

key[1]=0;
parent[1] = -1;

for(int i=1; i<n; i++) {
    int mini = INT_MAX;
    int u;

    for(int v=1; v<=n; v++) {
        if(mst[v] == false && key[v] < mini) {
            u = v;
            mini = key[v];
        }
    }
}
```

T.C. = $O(n^2)$

```
        mst[u] = true;
        for (auto it: adj[u]) {
            int v = it.first;
            int w = it.second;
            if (mst[v] == false && w < key[v]) {
                parent[v] = u;
                key[v] = w;
            }
        }
    }
    vector<pair<pair<int, int>, int>> result;
    for (int i=2; i<=n; i++) {
        result.push_back({{parent[i], i}, key[i]});
    }
    return result;
}
```

Disjoint Set

2 Important operations

↳ findParent() or findSet()

↳ Union() or UnionSet()

5 Disconnected components

findParent(1) = 1

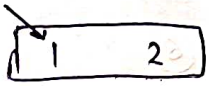
findParent(2) = 2

_____ (3) = 3

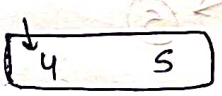
_____ (4) = 4

_____ (5) = 5

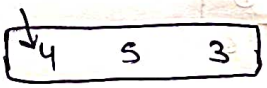
Union(1, 2)



Union(4, 5)



Union(3, 5)

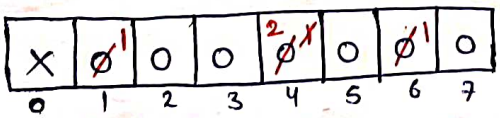


Union(1, 3)

Union By rank and Path compression

7 Disconnected components

rank



Union (1,2)

parent	Rank
1 → 1	0
2 → 2	0

(i) parent[2] = 1
(ii) rank[1]++

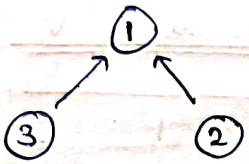
2 ka parent 1

Union (2,3)

2 → 1	1
3 → 3	0

rank[3] < rank[1]
parent[3] = 1

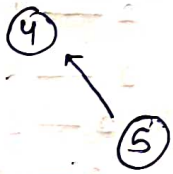
} Agar parent[] = 3 karenge to tree ki depth badegi



Union (4,5)

parent	Rank
4 → 4	0
5 → 5	0

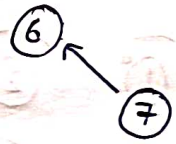
parent[5] = 4
rank[4]++



Union (6,7)

6 → 6	0
7 → 7	0

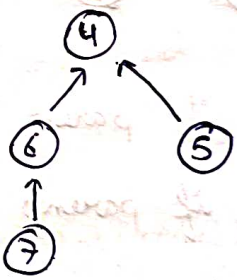
parent[7] = 6
rank[6]++



Union (5,6)

5 → 4	1
6 → 6	1

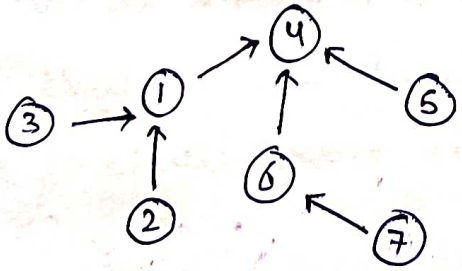
parent[6] = 4
rank[4]++



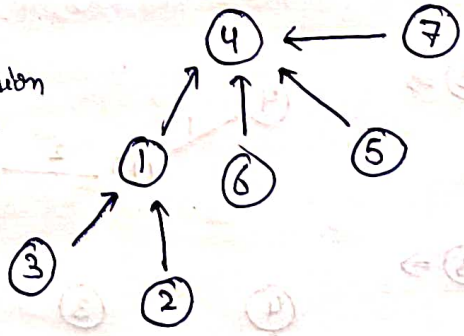
Union (3,7)

3 → 1	1
7 → 4	2

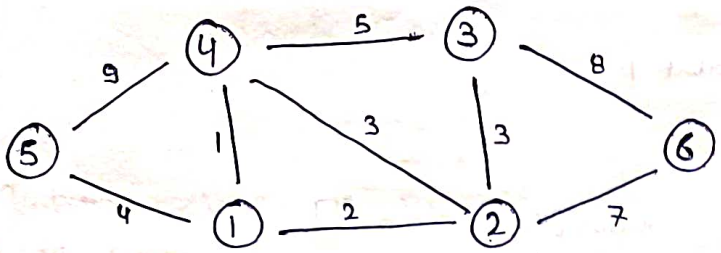
rank[1] < rank[4]
parent[1] = 4



Path compression



Kruskal's algorithm



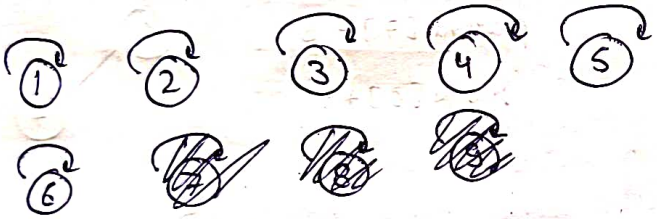
adj list ✗

linear Data structure ✓

1 — 2	2
1 — 4	1
1 — 5	4
4 — 5	9
4 — 3	5
2 — 4	3
2 — 3	3
2 — 6	7
3 — 6	8

Sorted ⟹

wt	u	v
1	1	4
2	1	2
3	2	3
3	2	4
4	1	5
5	4	3
7	2	6
8	3	6
9	4	5



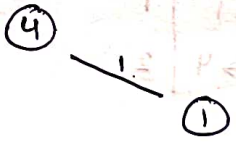
if parent is different then merge / union

if parent is same then ignore

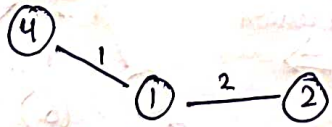
parent

① ⇒ 1 → 1

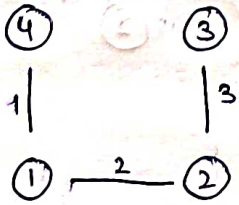
4 → 4



② ⇒

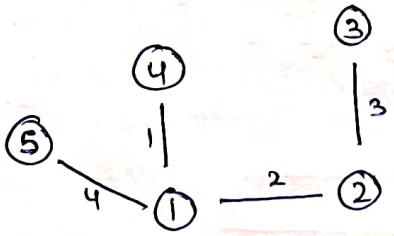


③ ⇒



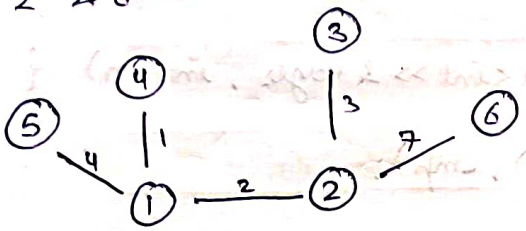
4 ⇒ 2 & 4 but in same component so ignore

5 ⇒ 1 & 5 but not in same component so merge



6 ⇒ 3 & 4 (same component) Ignore

7 ⇒ 2 & 6



} Minimum Spanning Tree

```
Void makeSet (vector<int> & parent, vector<int> & rank, int n) {
    for (int i=0; i<n; i++) {
        parent[i] = i;
        rank[i] = 0;
    }
}
int findParent (vector<int> & parent, int node) {
    if (parent[node] == node) {
        return node;
    }
    return parent[node] = findParent (parent, parent[node]);
}
                                        vector<int>& rank
Void unionSet (int u, int v, vector<int> & parent) {
    u = findParent (parent, u);
    v = findParent (parent, v);
    if (rank[u] < rank[v]) {
        parent[u] = v;
    }
    else if (rank[v] < rank[u]) {
        parent[v] = u;
    }
```

```
        else {
            parent[v] = u;
            rank[u]++;
        }
}

bool cmp(vector<int>& a, vector<int>& b) {
    return a[2] < b[2];
}

int minimumSpanningTree(vector<vector<int>>& edges, int n) {
    sort(edges.begin(), edges.end(), cmp);
    vector<int> parent(n);
    vector<int> rank(n);
    makeset(parent, rank, n);
    int minWeight=0;
    for (int i=0; i< edges.size(); i++) {
        int u= findParent(parent, edges[i][0]);
        int v= findParent(parent, edges[i][1]);
        int wt = edges[i][2];
        if (u != v) {
            unionset(u, v, parent, rank);
            minWeight += wt;
        }
    }
    return minWeight;
}
```

findParent → $O(4\alpha) \Rightarrow O(4) \Rightarrow \underline{O(1)}$

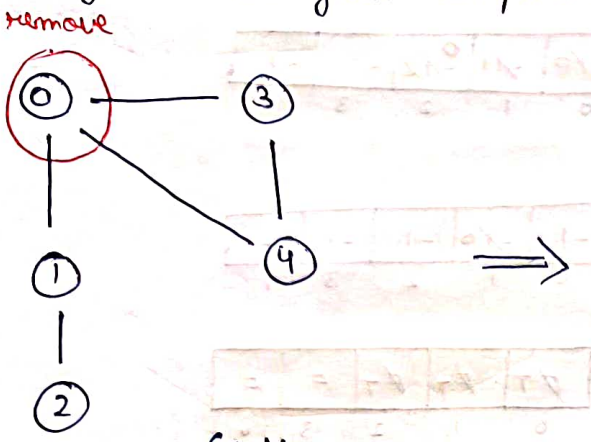
Unionset →

So, T.C. = $\underline{O(m \log m)}$

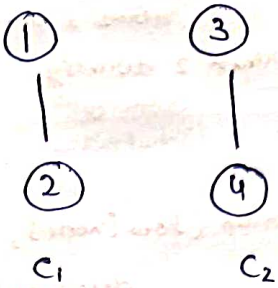
S.C. = $\underline{O(n)}$

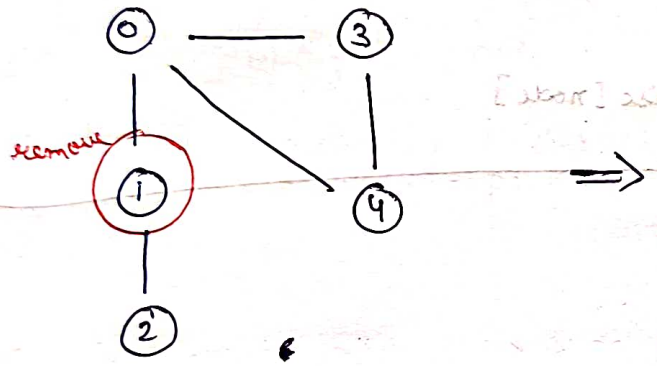
Articulation Point

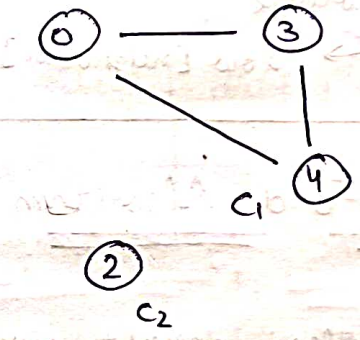
↳ Ek aisi node jisko mai remove kar du to mera graph 2 ya 2 se jada component me divide ho jayega



Graph



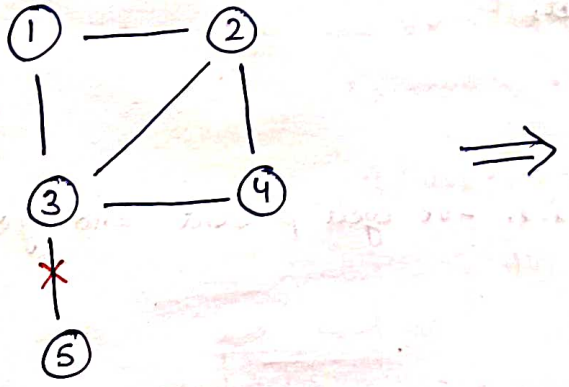




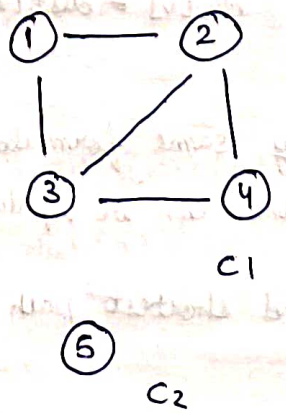
So, 0 and 1 are articulation points in this graph

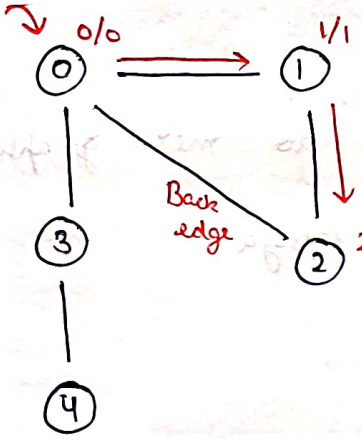
Bridge

↳ Ek aisi edge jisko agar remove karo to no of components badh jayenge



Graph





low time can be upgraded because back edge is present that means i can also reach 2 directly from 0

discovery

0	1	2	3	4
~~0~~	~~1~~	~~2~~	-1	-1

low

0	1	2	3	4
~~0~~	~~1~~	0 ~~2~~	-1	-1

parent

0	1	2	3	4
-1	~~0~~	-1	-1	-1

timer = ~~0~~ ~~1~~ 2

vis

0	1	2	3	4
~~F~~ T	~~F~~ T	~~F~~ T	F	F

low [node] = min (low [node], disc [neighbour])

if (neighbour == parent)
↳ ignore

To check bridge
↳ low [neighbour] > disc [node]

Bellman Ford Algorithm

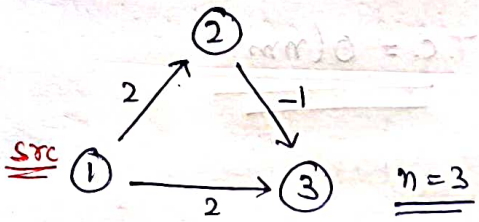
(Dijkstra's Algo doesn't work for -ve weights)

So, we use Bellman Ford Algo for that but the graph should ~~dont~~ not have -ve cycle present

① Apply given formula (n-1) times :- (on all edges)

```
if (dist[u] + wt. < dist[v]) {
    dis[v] = dist[u] + wt;
}
```

② 1 more time apply same formula
if distance can be updated i.e. -ve cycle present and you can't find shortest path



1 → 2 (2)
2 → 3 (-1)
1 → 3 (2)

0	∞ 2	∞ 1
1	2	3

(i) dist[1] + 2 < dist[2]
0 + 2 < ∞
2 < ∞

(ii) dist[1] + 2 < dist[2]
0 + 2 < 2
2 < 2

dist[2] + (-1) < dist[3]
2 - 1 < ∞
1 < ∞

dist[2] + (-1) < dist[3]
2 - 1 < 1
1 < 1

dist[1] + 2 < dist[3]
0 + 2 < 1
2 < 1 (False) So no updation

dist[1] + 2 < dist[3]
0 + 2 < 1
2 < 1

Now for checking -ve cycle

dist[1] + 2 < dist[2]
0 + 2 < 2
2 < 2 (F)

dist[2] + (-1) < dist[3]
2 - 1 < 1
1 < 1 (F)

dist[1] + 2 < dist[3]
0 + 2 < 1
2 < 1 (F)

i.e. -ve cycle not present

```
int bellmanFord(int n, int m, int src, int dest, vector<vector<int>>& edges) {
    vector<int> dist(n+1, 1e9);
    dist[src] = 0;
    for(int i=1; i<=n; i++) {
        for(int j=0; j<m; j++) {
            int u = edges[j][0];
            int v = edges[j][1];
            int wt = edges[j][2];
            if(dist[u] != 1e9 && ((dist[u] + wt) < dist[v])) {
                dist[v] = dist[u] + wt;
            }
        }
    }
    bool flag = 0;
```

```
for (int j=0; j<m; j++) {
    int u = edges[j][0];
    int v = edges[j][1];
    int wt = edges[j][2];
    if (dist[u] != 1e9 && (dist[u] + wt < dist[v])) {
        flag = 1;
    }
}
if (flag == 0) {
    return dist[dest];
}
return -1;
}
```

T.C. = O(nm)